Universidad de Ingeniería y Tecnología (UTEC)
CS3501 – Arquitectura de Computadoras

Proyecto 2: Procesador RISC-V superescalar Dual-Issue

Informe de la segunda entrega (E2)

Procesador Dual-Issue In-Order con dos vías de ejecución y recursos compartidos

Integrantes

Kenzo Shibuya
Surquislla Brisseth
Gabriel Díaz

Lima, 2026

Índice

1. Introducción **2**

2. Arquitectura Dual-Issue **2**
2.1. Organización general . . . 2
2.2. Datapath completo . . . 3
2.3. Controller . . . 4
2.4. Módulos modificados . . . 4

3. Issue Unit **5**
3.1. Condiciones de emisión . . . 5
3.2. Asignación de recursos . . . 5
3.3. Conservación y reagrupamiento . . . 6

4. Forwarding, hazards, stalls y control de branches **6**
4.1. Forwarding entre vías y etapas . . . 6
4.2. Load-Use . . . 7
4.3. Control de branches y actualización del PC . . . 7
4.4. Dos escrituras en el Register File . . . 8

5. Programas de prueba **8**
5.1. Metodología . . . 8
5.2. Programas de E2 . . . 9
5.3. Resultados en memoria . . . 9
5.4. P5: emisión simultánea de operaciones ALU . . . 9
P5: `prog5_dual_alu` . . . 9
5.5. P6: RAW entre candidatas y forwarding entre vías . . . 11
P6: `prog6_dual_raw` . . . 11
5.6. P7: puerto de memoria y Load-Use . . . 12
P7: `prog7_dual_mem` . . . 12
5.7. P8: branches en ambas posiciones . . . 14
P8: `prog8_dual_branch` . . . 14
5.8. P9: reagrupamiento y orden del programa . . . 17
P9: `prog9_dual_regroup` . . . 17

6. Comparación preliminar de ciclos e IPC **18**
6.1. Análisis . . . 19

7. Conclusiones **20**

1. Introducción

En la primera entrega (E1) se construyó un pipeline RV32I de cinco etapas con *forwarding*, *stalling* y *flushing*, y se incorporó la base de la arquitectura superescalar: Dual Fetch, Dual Decode, un Register File de cuatro lecturas y dos escrituras y los módulos de la segunda vía, con la emisión limitada a una instrucción por ciclo.

En esta segunda entrega el procesador pasa a ser *Dual-Issue In-Order*: en cada ciclo puede emitir dos instrucciones consecutivas del programa, una por vía. Para ello se completó la Issue Unit (detección de dependencias y conflictos, asignación de recursos y reagrupamiento de la instrucción no emitida), se extendió la Hazard Unit con *forwarding* entre vías y etapas, Load-Use desde cualquiera de las dos vías y *flush* selectivo, y se incorporaron los dos recursos compartidos que exige el enunciado: un único puerto de memoria de datos y una única unidad de resolución de saltos.

El informe describe la arquitectura y el datapath completo, la Issue Unit, el manejo de riesgos y saltos, los programas de prueba con sus resultados esperados y una comparación preliminar de ciclos e IPC frente al Single-Issue.

2. Arquitectura Dual-Issue

2.1. Organización general

El procesador mantiene las cinco etapas del pipeline (IF, ID, EX, MEM, WB) y las replica en dos vías. La vía 1 contiene siempre la instrucción más antigua del par y la vía 2 la siguiente en el orden del programa. La Tabla 1 resume qué recursos están duplicados y cuáles son compartidos.

Recurso	**Organización en E2**
Memoria de instrucciones	Una memoria con dos puertos de lectura: `rd0 = M[PC]` y `rd1 = M[PC+4]` (Dual Fetch).
PC	Un solo PC. Avanza `+8` si se emitió el par completo y `+4` si solo se emitió la vía 1 (`PCInc8F`); un salto tomado en EX tiene prioridad.
Registros IF/ID	Dos juegos (`InstrD/PCD/PCPlus4D` y `InstrD2/PCD2/PCPlus4D2`) con multiplexores de reagrupamiento a la entrada.
Unidad de control	Dos instancias de `controller` (`c` y `c2`), una por vía, sin cambios internos respecto a E1.
Register File	4 lecturas y 2 escrituras; ante dos escrituras al mismo registro gana la vía 2.
ALU	Dos ALU, una por vía.
Registros ID/EX, EX/MEM, MEM/WB	Duplicados; los de la vía 2 llevan el sufijo `2`.
Unidad de saltos	**Compartida**: un sumador `PCTargetE` y una señal `PCSrcE`.
Memoria de datos	**Compartida**: un solo puerto (dirección, dato y `WE`).
Hazard Unit	Una, que calcula el *forwarding* de los cuatro operandos, el Load-Use y los *flush*.
Issue Unit	Una, en ID: decide si la vía 2 se emite (`IssueSlot2D`).

Tabla 1: Recursos duplicados y compartidos del procesador Dual-Issue.

Como los dos recursos compartidos solo pueden atender a una instrucción por ciclo, la Issue Unit garantiza que nunca entren a EX dos instrucciones que los necesiten a la vez. Gracias a esto, la selección de qué vía usa cada recurso en EX y MEM se resuelve con un simple multiplexor, sin arbitraje adicional.

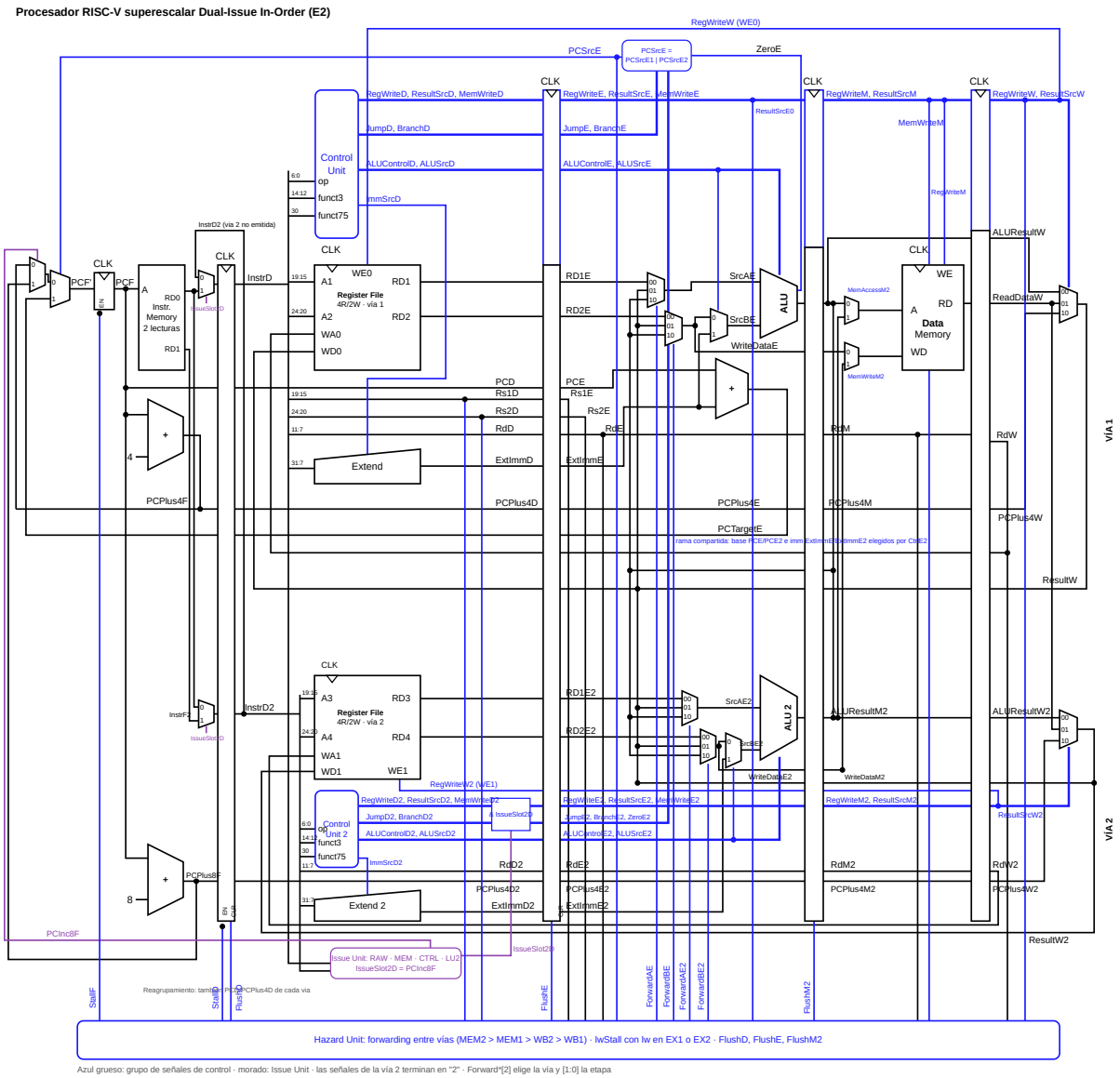


Figura 1: Datapath del procesador RISC-V Dual-Issue In-Order (E2). En azul, señales de control y de la Hazard Unit; en morado, la Issue Unit.

2.2. Datapath completo

La Figura 1 muestra el datapath Dual-Issue completo, con las unidades de control de cada vía simplificadas y separadas.

Los elementos nuevos respecto al datapath preliminar de E1 son:

- **Reagrupamiento IF/ID.** Dos multiplexores controlados por `IssueSlot2D` delante de los registros IF/ID eligen entre el par recién buscado (`InstrF`, `InstrF2`) y el par desplazado (`InstrD2`, `InstrF`). Lo mismo se hace con los PC asociados (Sección 3.3).
- **Multiplexor del PC secuencial.** `PCPlus4F` o `PCPlus8F` según `PCInc8F`, antes del multiplexor de salto.
- **Forwarding de la vía 2.** Dos multiplexores de cinco entradas (`ForwardAE2`, `ForwardBE2`) delante de la ALU 2. Los multiplexores de la vía 1 también pasan a cinco entradas, porque cada operando puede venir de MEM o WB de cualquiera de las dos vías.
- **Unidad de saltos compartida.** La base (`PCE` o `PCE2`) y el inmediato (`ImmExtE` o `ImmExtE2`) del sumador `PCTargetE` se eligen con `CtrlE2`, que vale 1 cuando la instrucción de control está en la vía 2.

- **Puerto de memoria compartido.** Dos multiplexores eligen la dirección y el dato de escritura de la vía que accede a memoria; `ReadData` llega a los multiplexores de resultado de ambas vías.
- **Flush de la vía 2 en EX/MEM** (`FlushM2`), para anular la instrucción más joven del par cuando el salto de la vía 1 se toma.

2.3. Controller

El decodificador no cambió internamente: se usan dos instancias idénticas de `controller` (`maindec` + `aludec`), una por vía. Se conserva la corrección de E1 en `maindec`: el código de operación `0000000` (burbuja o registro limpiado por *flush*) genera todas las señales de control en 0.

Lo que sí cambió es cómo entran al pipeline las señales de la vía 2. Antes del registro ID/EX se enmascaran con `IssueSlot2D`:

$$\texttt{RegWriteI2} = \texttt{RegWriteD2} \cdot \texttt{IssueSlot2D}, \qquad \texttt{MemWriteI2} = \texttt{MemWriteD2} \cdot \texttt{IssueSlot2D},$$
$$\texttt{JumpI2} = \texttt{JumpD2} \cdot \texttt{IssueSlot2D}, \qquad \texttt{BranchI2} = \texttt{BranchD2} \cdot \texttt{IssueSlot2D},$$

y además `ResultSrcI2` y `RdI2` se fuerzan a cero cuando la vía 2 no se emite. Así, una instrucción no emitida entra a EX2 como burbuja: no escribe registros ni memoria, no salta y no es vista como productora por la Hazard Unit. La instrucción no se pierde, porque en ese mismo flanco se copia a la vía 1 de ID.

2.4. Módulos modificados

Módulo	Cambios en E2
`issue_unit.v`	Nuevo contenido: condiciones RAW, MEM, CTRL y Load-Use de la vía 2; salidas `IssueSlot2D` y `PCInc8F`.
`hazard.v`	*Forwarding* de 3 bits para los cuatro operandos, Load-Use con el `lw` en EX1 o EX2, uso real de rs1/rs2 (`UsesRs1D`, `UsesRs2D`), `FlushM2`.
`datapath.v`	Reagrupamiento IF/ID, PC $+4/+8$, enmascarado de la vía 2, *forwarding* de la vía 2, unidad de saltos y puerto de memoria compartidos, `FlushM2` en EX/MEM de la vía 2.
`riscvpipe.v`	Conexión de la Issue Unit, la Hazard Unit extendida y las señales de la vía 2.
`imem.v`	Dos lecturas (`rd0`, `rd1`) y parámetro `MEM_FILE`; posiciones vacías inicializadas a 0 (burbuja).
`regfile.v`	4R/2W con escritura en flanco de bajada; la escritura de la vía 2 se aplica después y tiene prioridad.
`top.v`	Conexión de las dos lecturas de `imem` y del puerto único de `dmem`.
`controller.v`, `maindec.v`, `aludec.v`, `alu.v`, `extend.v`, `dmem.v`	Sin cambios respecto a E1.

Tabla 2: Módulos modificados en la segunda entrega.

3. Issue Unit

La Issue Unit trabaja en ID sobre el par (`InstrD`, `InstrD2`), esto para determinar si este par de instrucciones puede emitirse simultáneamente hacia la etapa de Ejecución. La instrucción de la vía 1 se emite siempre que el pipeline no esté detenido; la unidad solo decide si la vía 2 la acompaña. Su salida es

$$\texttt{IssueSlot2D} = \overline{\text{RAW} + \text{MEM} + \text{CTRL} + \text{LU2}}, \qquad \texttt{PCInc8F} = \texttt{IssueSlot2D}.$$

3.1. Condiciones de emisión

Condición	Expresión	Motivo
RAW	`RegWriteD` · (`RdD` $\neq$ 0) · [(`UsesRs1D2` · `Rs1D2 = RdD`) + (`UsesRs2D2` · `Rs2D2 = RdD`)]	La vía 2 lee el registro que escribe la vía 1 del mismo par. Las dos llegarían a EX en el mismo ciclo y el resultado de la vía 1 aún no existe; no hay *forwarding* posible dentro de EX.
MEM	(`MemWriteD` + `ResultSrcD[0]`) · (`MemWriteD2` + `ResultSrcD2[0]`)	Las dos instrucciones son `lw` o `sw`: solo hay un puerto de memoria de datos.
CTRL	(`BranchD` + `JumpD`) · (`BranchD2` + `JumpD2`) + `JumpD`	Dos instrucciones de control compiten por la única unidad de saltos. Además, si la vía 1 es `jal`, la vía 2 está siempre en el camino incorrecto y no tiene sentido emitirla.
LU2	`lw` en EX1 o EX2 con `Rd` igual a un registro que lee la vía 2	Load-Use de la vía 2. Si la vía 1 no depende del `lw`, no hace falta detener todo el pipeline: basta con no emitir la vía 2 y reagruparla; en el ciclo siguiente recibirá el dato por *forwarding* desde WB.

Tabla 3: Condiciones que impiden emitir la vía 2.

Para evitar falsos conflictos, la unidad solo compara los campos rs1/rs2 que la instrucción realmente lee:

$$\texttt{UsesRs1D2} = \overline{\texttt{JumpD2}} \cdot (\texttt{op} \neq 0), \qquad \texttt{UsesRs2D2} = (\texttt{op} = \text{R}) + \texttt{MemWriteD2} + \texttt{BranchD2}.$$

En un `addi` o un `lw`, los bits 24:20 son parte del inmediato; sin esta precaución, un `addi x5, x0, 3` detrás de una instrucción que escribe `x3` dejaría de emitirse en pareja sin motivo.

Las dependencias que *no* impiden la emisión dual son WAR y WAW dentro del par. En el WAR la vía 2 escribe un registro que la vía 1 lee, pero la vía 1 ya lo leyó en ID. En el WAW ambas escriben el mismo registro en el mismo ciclo de WB y el Register File da prioridad a la vía 2, que es la más joven, por lo que el valor final es el del orden del programa (P6 y P9 lo verifican con `x7` y `x10`).

3.2. Asignación de recursos

Una vez emitido el par, cada recurso compartido queda asignado a la única instrucción que lo usa:

- **Memoria de datos.** En MEM, `MemAccessM2 = MemWriteM2 + ResultSrcM2[0]`. Si es 1, la dirección es `ALUResultM2`; si no, `ALUResultM`. El dato de escritura es `WriteDataM2` si `MemWriteM2` y `WriteDataM` en otro caso, y `MemWrite = MemWriteM + MemWriteM2`. Las combinaciones `lw`/`sw` + ALU se emiten juntas en cualquier orden.
- **Unidad de saltos.** En EX, `CtrlE2 = BranchE2 + JumpE2` elige la base y el inmediato del sumador `PCTargetE`. Cada vía calcula su propia condición, `PCSrcE1 = BranchE·ZeroE + JumpE` y `PCSrcE2 = BranchE2·ZeroE2 + JumpE2`, y el salto se toma si `PCSrcE = PCSrcE1 + PCSrcE2`. Un salto combinado con una ALU, en cualquier orden, se emite en pareja.

- **Puertos de escritura del Register File.** No son compartidos: cada vía tiene el suyo (`WE0` y `WE1`).

Como la Issue Unit impide dos accesos a memoria o dos saltos en el mismo par, nunca se activan a la vez `MemAccessM2` y un acceso de la vía 1, ni `PCSrcE1` y `PCSrcE2` por instrucciones del mismo par.

3.3. Conservación y reagrupamiento

Cuando la vía 2 no se emite, esa instrucción no puede descartarse ni volver a buscarse dos veces. El diseño la conserva desplazándola a la vía 1 en el siguiente flanco y completando el par con la instrucción que estaba en IF en la posición 1:

Siguiente valor	`IssueSlot2D = 1` (par completo)	`IssueSlot2D = 0` (solo vía 1)
`InstrD` / `PCD` / `PCPlus4D`	`InstrF` / `PCF` / `PCPlus4F`	`InstrD2` / `PCD2` / `PCPlus4D2`
`InstrD2` / `PCD2` / `PCPlus4D2`	`InstrF2` / `PCPlus4F` / `PCPlus8F`	`InstrF` / `PCF` / `PCPlus4F`
`PCF` (sin salto)	`PCF + 8`	`PCF + 4`

Tabla 4: Multiplexores de reagrupamiento a la entrada de IF/ID.

El diseño mantiene en todo momento el invariante `PCF` = `PCD2`+4: la instrucción en IF posición 1 es siempre la siguiente a la que está en la vía 2 de ID. Si se emite el par, entran las dos instrucciones buscadas y el PC avanza 8; si solo se emite la vía 1, la vía 2 pasa a ser la vía 1, la instrucción de `PCF` pasa a la vía 2 y el PC avanza solo 4, de modo que la instrucción de `PCF+4` se vuelve a buscar en la posición 1 en el ciclo siguiente. Ninguna instrucción se pierde ni se duplica, y la vía 1 sigue conteniendo siempre la más antigua, por lo que el orden del programa se respeta. Los pares pueden empezar en cualquier dirección múltiplo de 4, no solo en múltiplos de 8.

La interacción con los otros mecanismos es la siguiente: con `StallD` los registros IF/ID conservan el par y la decisión se recalcula en el ciclo siguiente; con `FlushD` (salto tomado) ambos registros se limpian y el reagrupamiento queda sin efecto, porque el par pertenecía al camino incorrecto.

4. Forwarding, hazards, stalls y control de branches

4.1. Forwarding entre vías y etapas

Cada uno de los cuatro operandos de EX (`SrcAE`, `SrcBE`, `SrcAE2`, `SrcBE2`) puede tomarse del Register File o de las dos instrucciones que están en MEM y las dos que están en WB. Los selectores tienen 3 bits: el bit 2 indica la vía productora y los bits 1:0 la etapa, igual que en E1.

Código	**Fuente**	**Valor**
`000`	Register File	`RD1E` / `RD2E` (o `RD1E2` / `RD2E2`)
`010`	MEM vía 1	`ForwardDataM1`
`110`	MEM vía 2	`ForwardDataM2`
`001`	WB vía 1	`ResultW`
`101`	WB vía 2	`ResultW2`

Tabla 5: Códigos de `ForwardAE`, `ForwardBE`, `ForwardAE2` y `ForwardBE2`.

La condición para cada fuente es la de E1 (registro igual, `RegWrite` activo y registro distinto de `x0`) y la prioridad es

$$\text{MEM2} > \text{MEM1} > \text{WB2} > \text{WB1}.$$

MEM tiene prioridad sobre WB porque contiene un productor más reciente. Dentro de una misma etapa, la vía 2 tiene prioridad porque es la más joven del par: si el par anterior escribió dos veces el

mismo registro (WAW), el valor correcto es el de la vía 2. El programa P6 ejercita las dos prioridades con `x7`.

En MEM, el valor reenviado es

$$\texttt{ForwardDataM} = (\texttt{ResultSrcM} = 10)\ ?\ \texttt{PCPlus4M} : \texttt{ALUResultM}.$$

Así, el registro de enlace de un `jal` se reenvía con su valor correcto (`PC+4`) y no con la dirección calculada por la ALU. Para un `lw` en MEM no se reenvía nada útil desde esa etapa; ese caso lo cubren el Load-Use y la Issue Unit.

El *forwarding* dentro de un mismo par (vía 1 → vía 2 en EX) no existe: ese es justamente el caso RAW que la Issue Unit separa.

4.2. Load-Use

Un `lw` obtiene el dato al final de MEM, así que una instrucción que lo usa no puede estar en EX en el ciclo inmediatamente posterior. Se distinguen dos casos según la vía del consumidor:

- **Consumidor en la vía 1 de ID.** La Hazard Unit detiene el pipeline:

$$\begin{aligned}
\texttt{lw_in_E1} &= \texttt{ResultSrcE[0]} \cdot (\texttt{RdE} \neq 0),\\
\texttt{lw_in_E2} &= \texttt{ResultSrcE2[0]} \cdot (\texttt{RdE2} \neq 0) \cdot \overline{\texttt{PCSrcE1}},\\
\texttt{lwStall} &= \texttt{lw_in_E1} \cdot \text{usa}(\texttt{RdE}) + \texttt{lw_in_E2} \cdot \text{usa}(\texttt{RdE2}),
\end{aligned}$$

  donde *usa*(r) compara r con `Rs1D`/`Rs2D` solo si la instrucción los lee (`UsesRs1D`, `UsesRs2D`). El término $\overline{\texttt{PCSrcE1}}$ evita un stall innecesario cuando el `lw` de la vía 2 va a ser anulado por un salto tomado en la vía 1.

- **Consumidor solo en la vía 2 de ID.** No se detiene el pipeline: la Issue Unit no emite la vía 2 (condición LU2). La vía 1 avanza, la vía 2 se reagrupa como vía 1 del par siguiente y, un ciclo después, el `lw` está en WB y su dato llega por *forwarding*. El costo es una emisión parcial en lugar de un ciclo completo perdido.

Si el consumidor está en la vía 1, el stall afecta al par entero: `StallF` y `StallD` congelan PC e IF/ID y `FlushE` inserta burbujas en las dos vías de EX.

4.3. Control de branches y actualización del PC

Los saltos se resuelven en EX, como en E1, con la unidad compartida descrita en la sección anterior. El PC se actualiza con la siguiente prioridad:

$$\texttt{PCNextF} = \begin{cases} \texttt{PCTargetE} & \text{si } \texttt{PCSrcE} = 1,\\ \texttt{PCF} + 8 & \text{si } \texttt{PCInc8F} = 1,\\ \texttt{PCF} + 4 & \text{en otro caso,} \end{cases}$$

y el registro del PC no se carga si `StallF` está activo. Un salto tomado tiene prioridad sobre un Load-Use ($\texttt{StallF = StallD = lwStall} \cdot \overline{\texttt{PCSrcE}}$), porque las instrucciones detenidas pertenecen al camino incorrecto.

Cuando el salto se toma, se anulan las instrucciones más jóvenes que él:

Caso	Acción
Salto tomado en la vía 1	`FlushD` limpia el par de ID, `FlushE` el par que entraría a EX y `FlushM2 = PCSrcE1` anula la vía 2 del propio par, que es más joven que el salto y avanzaría a MEM.
Salto tomado en la vía 2	`FlushD` y `FlushE` como antes. La vía 1 del mismo par es más antigua que el salto y **se conserva**: `FlushM2` no se activa.
`jal` en la vía 1 (ID)	La Issue Unit no emite la vía 2 (CTRL), porque siempre sería camino incorrecto.

Tabla 6: Flushing selectivo según la posición del salto.

La penalización de un salto tomado sigue siendo de 2 ciclos, pero ahora cada ciclo perdido equivale a dos ranuras de emisión: un salto tomado descarta hasta cuatro instrucciones (las dos de ID y las dos que se buscaban en IF), más la vía 2 de su propio par si el salto está en la vía 1.

4.4. Dos escrituras en el Register File

Ambas vías escriben en el flanco de bajada, de modo que una instrucción en ID lee en el mismo ciclo el valor que escriben las dos instrucciones en WB (como en E1, no se necesita *forwarding* desde WB hacia ID). La escritura de la vía 2 se aplica después de la de la vía 1 dentro del mismo bloque `always`, por lo que, si ambas escriben el mismo registro, prevalece la vía 2. Ninguna escritura afecta a `x0`.

5. Programas de prueba

5.1. Metodología

Cada programa se ensambló a código máquina (`.mem`) y se simuló en el procesador E2 con el parámetro `MEM_FILE`.

Además de los cinco programas de E2, se ejecutaron en el procesador Dual-Issue los cuatro programas de E1 (P1–P4), con los mismos resultados que en el Single-Issue.

5.2. Programas de E2

Programa	Requisito (3.2)	Qué ejercita
P5 `prog5_dual_alu`	Dos ALU independientes emitidas juntas	Ocho pares ALU seguidos emitidos en pareja; todas las operaciones (`add`, `sub`, `and`, `or`, `slt` y variantes I). Después, diez `sw` que el puerto único emite de a uno.
P6 `prog6_dual_raw`	RAW entre candidatas y *forwarding* entre vías/etapas	RAW dentro del par y reagrupamiento; MEM1→EX2, MEM2→EX1, WB1→EX2, WB2→EX1; WAW en el par y prioridad MEM2 sobre MEM1 y WB2 sobre WB1.
P7 `prog7_dual_mem`	Load-Use y conflictos del puerto de memoria	`sw`+ALU, ALU+`sw`, `lw`+ALU y ALU+`lw` en pareja; dos `sw` y `lw`+`sw` separados; Load-Use desde EX2 (stall), Load-Use en la vía 2 (LU2) y `lw` con su consumidor en el mismo par.
P8 `prog8_dual_branch`	Branches en primera o segunda posición y *flushing*	`beq` tomado y no tomado en cada vía, dos saltos en el par, `jal` en cada vía con su registro de enlace y un bucle con salto hacia atrás desde la vía 2.
P9 `prog9_dual_regroup`	Dos escrituras en el RF; reagrupamiento sin pérdidas ni duplicados	Cadena de dependencias sobre `x1` que fuerza emisiones parciales consecutivas, en direcciones pares e impares; contador de instrucciones intercaladas; dos escrituras al mismo registro.

Tabla 7: Programas de prueba de la segunda entrega.

5.3. Resultados en memoria

Dirección	P5	P6	P7	P8	P9
64	30	12	11	15	135
68	35	15	22	76 (`0x4C`)	6
72	10	−9	33	92 (`0x5C`)	12
76	−1	2	44	6	2
80	1	4	11	0	–
84	1	17	22	–	–
88	4	6	33	–	–
92	15	−11	33	–	–
96	65	8	66	–	–
100	−8	–	44	–	–
104	–	–	55	–	–
108	–	–	11	–	–
112	–	–	22	–	–
116	–	–	55	–	–
120	–	–	66	–	–
124	–	–	77	–	–
`done`	`0x68`	`0x60`	`0x98`	`0x94`	`0x78`

Tabla 8: Contenido final esperado de la memoria de datos (decimal). Coincide con el simulador de referencia en E1 y en E2.

5.4. P5: emisión simultánea de operaciones ALU

`prog5_dual_alu`

c	t(ns)	Vía 1 en ID	Vía 2 en ID	Decisión	Eventos	Forwarding (par en EX)
0	30	00 addi x1, x0, 10	04 addi x2, x0, 20	DUAL		
1	40	08 addi x3, x0, 30	0c addi x4, x0, -5	DUAL		
2	50	10 addi x20, x0, 64	14 addi x5, x0, 7	DUAL		
3	60	18 add x6, x1, x2	1c sub x7, x3, x4	DUAL		
4	70	20 and x8, x1, x3	24 or x9, x2, x4	DUAL		A2=WB1 B2=WB2
5	80	28 slt x10, x4, x1	2c slti x11, x3, 100	DUAL		
6	90	30 andi x12, x2, 12	34 ori x13, x1, 5	DUAL		
7	100	38 add x14, x6, x7	3c sub x15, x9, x5	DUAL		
8	110	40 sw x6, 0(x20)	44 sw x7, 4(x20)	Vía 1 (MEM)		
9	120	44 sw x7, 4(x20)	48 sw x8, 8(x20)	Vía 1 (MEM)		
10	130	48 sw x8, 8(x20)	4c sw x9, 12(x20)	Vía 1 (MEM)		
11	140	4c sw x9, 12(x20)	50 sw x10, 16(x20)	Vía 1 (MEM)		
12	150	50 sw x10, 16(x20)	54 sw x11, 20(x20)	Vía 1 (MEM)		
13	160	54 sw x11, 20(x20)	58 sw x12, 24(x20)	Vía 1 (MEM)		
14	170	58 sw x12, 24(x20)	5c sw x13, 28(x20)	Vía 1 (MEM)		
15	180	5c sw x13, 28(x20)	60 sw x14, 32(x20)	Vía 1 (MEM)		
16	190	60 sw x14, 32(x20)	64 sw x15, 36(x20)	Vía 1 (MEM)		
17	200	64 sw x15, 36(x20)	68 done: beq x0, x0, done	DUAL		
18	210	6c (burbuja)	70 (burbuja)	-	PCSrc FlushD	
19	220	00 (burbuja)	00 (burbuja)	-		

Tabla 9: Traza de ejecución de P5

Los ocho primeros pares (ciclos 0–7, de 30 a 100 ns) son instrucciones ALU sin dependencias dentro del par, y los ocho se emiten juntos: `IssueSlot2D` y `PCInc8F` permanecen en 1 y `PCF` avanza de 8 en 8. Las dependencias con pares anteriores se resuelven por *forwarding* o por el Register File; por ejemplo, en el ciclo 4 el par `add x6, x1, x2 | sub x7, x3, x4` está en EX y la vía 2 recibe `x3` desde WB1 y `x4` desde WB2. En cada ciclo de WB del tramo se activan a la vez `RegWriteW` y `RegWriteW2`, es decir, dos escrituras por ciclo en el Register File. A partir del ciclo 8, los diez `sw` se emiten de a uno por el conflicto MEM, y el último `sw` se empareja con el `beq` final (memoria + salto no compiten).

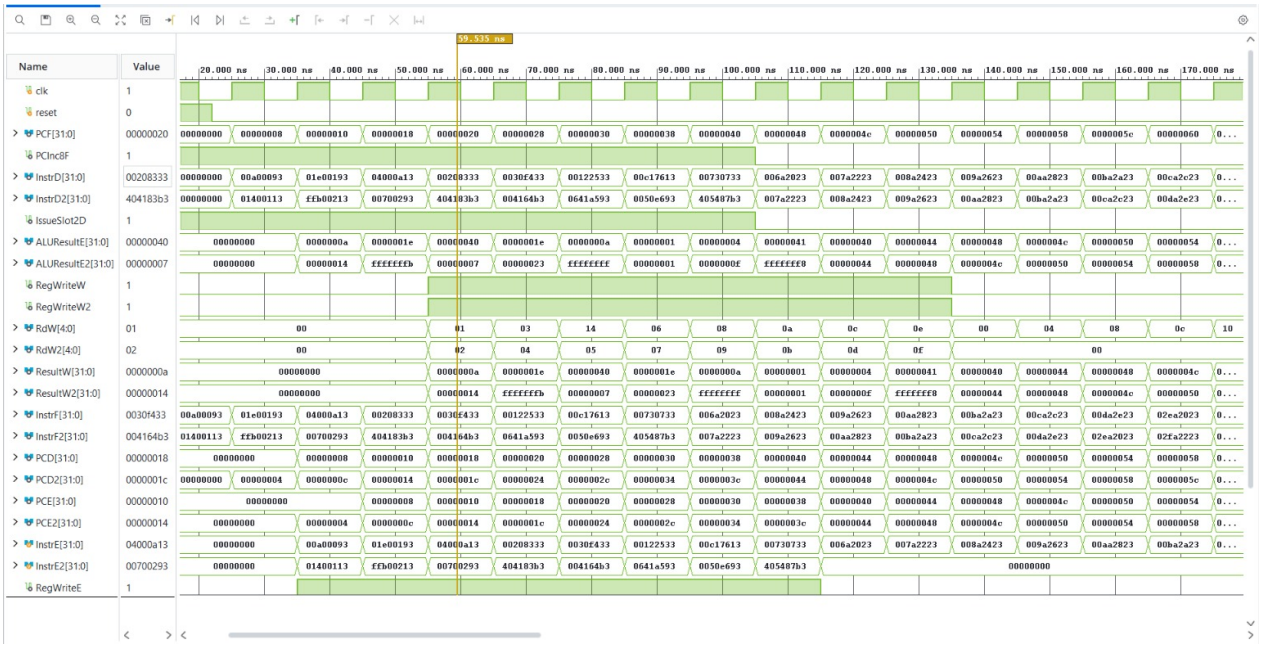


Figura 2: P5, 20–120 ns: emisión dual continua. Señales: `PCF`, `PCInc8F`, `InstrD`, `InstrD2`, `IssueSlot2D`, `ALUResultE`, `ALUResultE2`, `RegWriteW`, `RegWriteW2`, `RdW`, `RdW2`, `ResultW`, `ResultW2`.

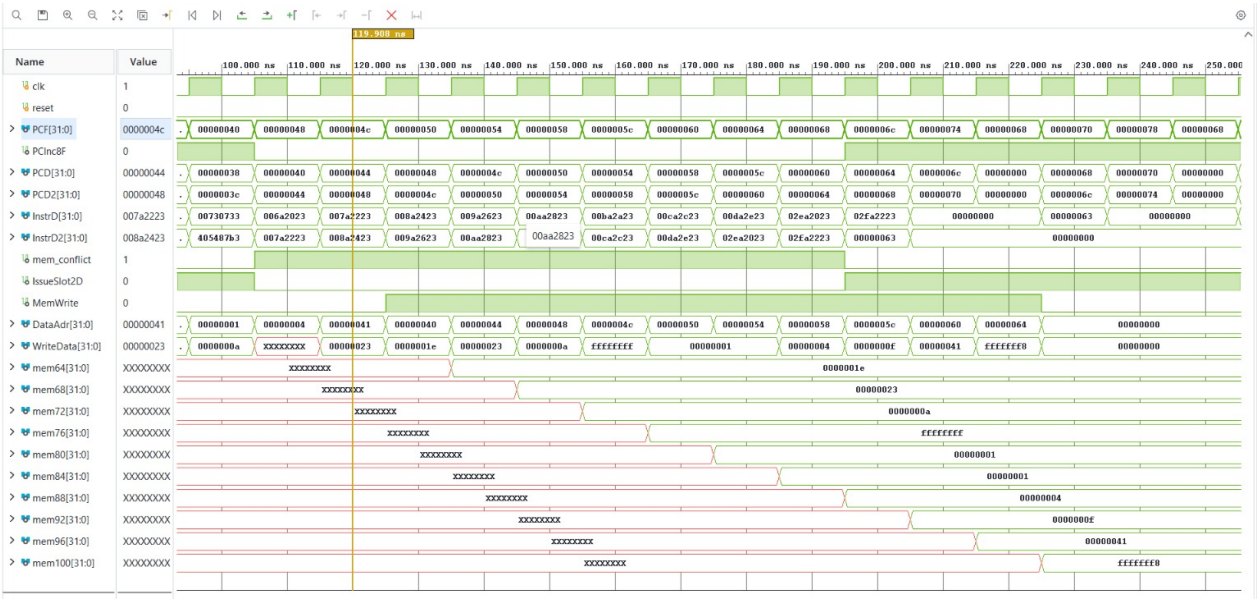


Figura 3: P5, 100–220 ns: secuencia de `sw` con conflicto de memoria. Señales: `PCF`, `InstrD`, `InstrD2`, `mem_conflict`, `IssueSlot2D`, `MemWrite`, `DataAdr`, `WriteData`.

5.5. P6: RAW entre candidatas y forwarding entre vías

`prog6_dual_raw`

c	t(ns)	Vía 1 en ID	Vía 2 en ID	Decisión	Eventos	Forwarding (par en EX)
0	30	`00 addi x20, x0, 64`	`04 addi x1, x0, 5`	DUAL		
1	40	`08 addi x2, x0, 7`	`0c add x3, x1, x2`	Vía 1 (RAW)		
2	50	`0c add x3, x1, x2`	`10 addi x4, x0, 3`	DUAL		
3	60	`14 add x5, x3, x4`	`18 sub x6, x4, x3`	DUAL		A1=WB2 B1=MEM1
4	70	`1c addi x7, x0, 1`	`20 addi x7, x0, 2`	DUAL		A1=MEM1 B1=MEM2 A2=MEM2 B2=MEM1
5	80	`24 add x8, x7, x7`	`28 addi x9, x0, 4`	DUAL		
6	90	`2c add x10, x7, x5`	`30 add x11, x8, x7`	DUAL		A1=MEM2 B1=MEM2
7	100	`34 sub x12, x11, x10`	`38 add x13, x9, x8`	DUAL		A1=WB2 A2=MEM1 B2=WB2
8	110	`3c sw x3, 0(x20)`	`40 sw x5, 4(x20)`	Vía 1 (MEM)		A1=MEM2 B1=MEM1 A2=WB2 B2=WB1
9	120	`40 sw x5, 4(x20)`	`44 sw x6, 8(x20)`	Vía 1 (MEM)		
10	130	`44 sw x6, 8(x20)`	`48 sw x7, 12(x20)`	Vía 1 (MEM)		
11	140	`48 sw x7, 12(x20)`	`4c sw x8, 16(x20)`	Vía 1 (MEM)		
12	150	`4c sw x8, 16(x20)`	`50 sw x10, 20(x20)`	Vía 1 (MEM)		
13	160	`50 sw x10, 20(x20)`	`54 sw x11, 24(x20)`	Vía 1 (MEM)		
14	170	`54 sw x11, 24(x20)`	`58 sw x12, 28(x20)`	Vía 1 (MEM)		
15	180	`58 sw x12, 28(x20)`	`5c sw x13, 32(x20)`	Vía 1 (MEM)		
16	190	`5c sw x13, 32(x20)`	`60 done: beq x0, x0, done`	DUAL		
17	200	`64 (burbuja)`	`68 (burbuja)`	-	PCSrc FlushD	
18	210	`00 (burbuja)`	`00 (burbuja)`	-		

Tabla 10: Traza de ejecución de P6

En el ciclo 1 (40 ns) el par `addi x2, x0, 7 | add x3, x1, x2` tiene RAW sobre `x2`: la Issue Unit emite solo la vía 1, `PCF` avanza 4 y en el ciclo 2 el `add` aparece en la vía 1 junto con `addi x4`, ya como par completo. Cuando ese `add` llega a EX (ciclo 3) toma `x2` de MEM1 y `x1` de WB2.

Los pares siguientes ejercitan todos los caminos cruzados: en el ciclo 4, `add x5, x3, x4` recibe `x4` desde MEM2 (vía 2 → vía 1) y `sub x6, x4, x3` recibe `x3` desde MEM1 (vía 1 → vía 2). El par `addi x7, x0, 1 | addi x7, x0, 2` se emite junto (WAW no es conflicto); en el ciclo 6, `add x8, x7, x7` encuentra `x7` en MEM1 y en MEM2 a la vez y elige MEM2 ($x8 = 4$), y en el ciclo 7 `add x10, x7, x5` encuentra `x7` en WB1 y WB2 y elige WB2. El Register File termina con `x7 = 2`.

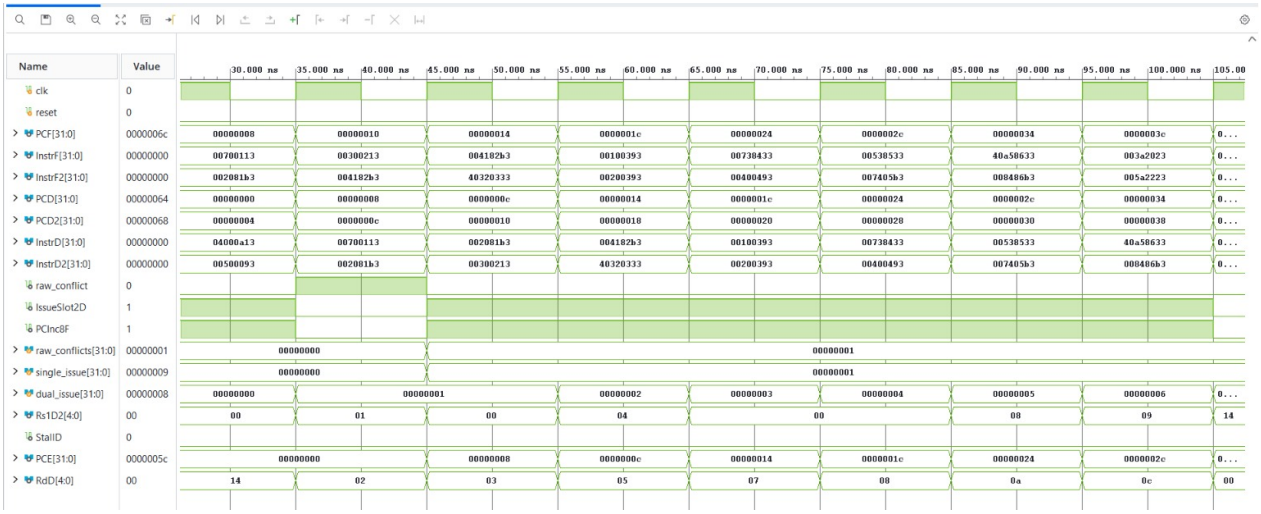


Figura 4: P6, 30–80 ns: RAW dentro del par y reagrupamiento. Señales: `PCF`, `InstrF`, `InstrF2`, `InstrD`, `InstrD2`, `PCD`, `PCD2`, `raw_hazard`, `IssueSlot2D`, `PCInc8F`.

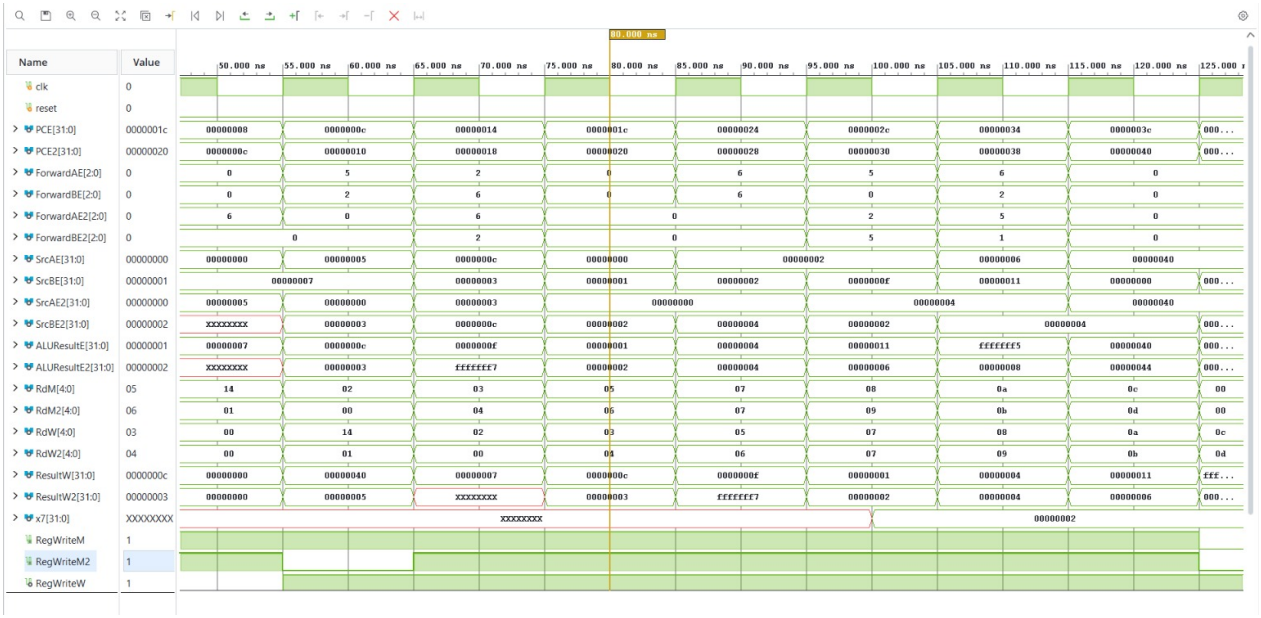


Figura 5: P6, 50–120 ns: *forwarding* entre vías y prioridades. Señales: `ForwardAE`, `ForwardBE`, `ForwardAE2`, `ForwardBE2`, `SrcAE`, `SrcBE`, `SrcAE2`, `SrcBE2`, `ALUResultE`, `ALUResultE2`, `RdM`, `RdM2`, `RdW`, `RdW2`.

5.6. P7: puerto de memoria y Load-Use

`prog7_dual_mem`

c	t(ns)	Vía 1 en ID	Vía 2 en ID	Decisión	Eventos	Forwarding (par en EX)
0	30	00 addi x20, x0, 64	04 addi x1, x0, 11	DUAL		
1	40	08 addi x2, x0, 22	0c addi x3, x0, 33	DUAL		
2	50	10 sw x1, 0(x20)	14 addi x4, x0, 44	DUAL		B2=MEM2
3	60	18 addi x5, x0, 55	1c sw x2, 4(x20)	DUAL		A1=WB1 B1=WB2
4	70	20 sw x3, 8(x20)	24 sw x4, 12(x20)	Vía 1 (MEM)		B2=WB1
5	80	24 sw x4, 12(x20)	28 addi x6, x0, 66	DUAL		
6	90	2c lw x7, 0(x20)	30 addi x8, x0, 77	DUAL		
7	100	34 addi x9, x0, 88	38 lw x10, 4(x20)	DUAL		
8	110	3c add x11, x10, x1	40 addi x12, x0, 1	DUAL	STALL	
9	120	3c add x11, x10, x1	40 addi x12, x0, 1	DUAL		
10	130	44 lw x13, 8(x20)	48 addi x14, x0, 2	DUAL		A1=WB2

c	t(ns)	Vía 1 en ID	Vía 2 en ID	Decisión	Eventos	Forwarding (par en EX)
11	140	4c addi x15, x0, 3	50 add x16, x13, x13	Vía 1 (LU2)		
12	150	50 add x16, x13, x13	54 addi x22, x0, 5	DUAL		
13	160	58 lw x17, 12(x20)	5c add x18, x17, x1	Vía 1 (RAW)		A1=WB1 B1=WB1
14	170	5c add x18, x17, x1	60 lw x19, 0(x20)	DUAL	STALL	
15	180	5c add x18, x17, x1	60 lw x19, 0(x20)	DUAL		
16	190	64 lw x21, 4(x20)	68 sw x7, 16(x20)	Vía 1 (MEM)		A1=WB1
17	200	68 sw x7, 16(x20)	6c sw x10, 20(x20)	Vía 1 (MEM)		
18	210	6c sw x10, 20(x20)	70 sw x11, 24(x20)	Vía 1 (MEM)		
19	220	70 sw x11, 24(x20)	74 sw x13, 28(x20)	Vía 1 (MEM)		
20	230	74 sw x13, 28(x20)	78 sw x16, 32(x20)	Vía 1 (MEM)		
21	240	78 sw x16, 32(x20)	7c sw x17, 36(x20)	Vía 1 (MEM)		
22	250	7c sw x17, 36(x20)	80 sw x18, 40(x20)	Vía 1 (MEM)		
23	260	80 sw x18, 40(x20)	84 sw x19, 44(x20)	Vía 1 (MEM)		
24	270	84 sw x19, 44(x20)	88 sw x21, 48(x20)	Vía 1 (MEM)		
25	280	88 sw x21, 48(x20)	8c sw x5, 52(x20)	Vía 1 (MEM)		
26	290	8c sw x5, 52(x20)	90 sw x6, 56(x20)	Vía 1 (MEM)		
27	300	90 sw x6, 56(x20)	94 sw x8, 60(x20)	Vía 1 (MEM)		
28	310	94 sw x8, 60(x20)	98 done: beq x0, x0, done	DUAL		
29	320	9c (burbuja)	a0 (burbuja)	-	PCSrc FlushD	
30	330	00 (burbuja)	00 (burbuja)	-		

Tabla 11: Traza de ejecución de P7

Los ciclos 2 y 3 muestran `sw`+ALU y ALU+`sw` emitidos en pareja; en el ciclo 3 el `sw` está en la vía 2, así que en MEM se activa `MemAccessM2` y la dirección sale de `ALUResultM2`. En el ciclo 4 dos `sw` seguidos producen conflicto MEM y el segundo se reagrupa con `addi x6`. En los ciclos 6 y 7, `lw` se combina con una ALU en ambos órdenes.

En el ciclo 8 (110 ns) el `add x11, x10, x1` está en la vía 1 de ID y el `lw x10` en EX2: `lwStall` se activa, el par se repite en el ciclo 9 y el dato llega desde WB2. En el ciclo 11 (140 ns) el consumidor `add x16, x13, x13` está en la vía 2 y el `lw x13` en EX1: no hay stall, la Issue Unit indica LU2, emite solo `addi x15` y en el ciclo 12 el `add` ya está en la vía 1. En el ciclo 13 el `lw x17` y su consumidor están en el mismo par (RAW): se separan y en el ciclo 14 se produce el stall normal. P7 tiene en total 2 stalls, uno menos de los que tendría si el caso LU2 se resolviera también con stall.

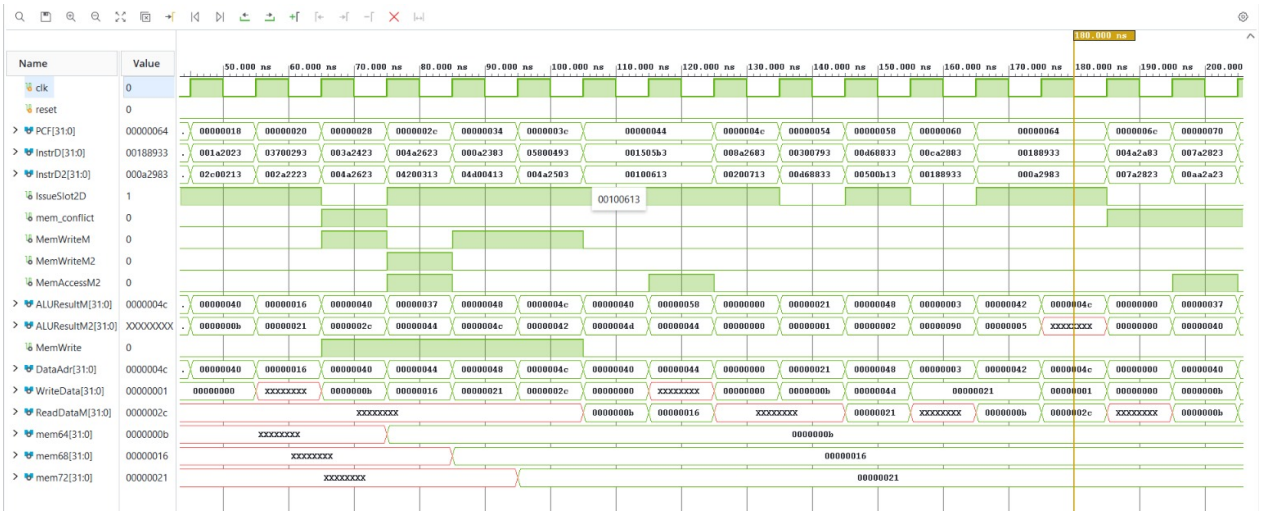


Figura 6: P7, 50–130 ns: asignación del puerto único de memoria. Señales: `InstrD`, `InstrD2`, `IssueSlot2D`, `MemWriteM`, `MemWriteM2`, `MemAccessM2`, `DataAdr`, `WriteData`, `MemWrite`, `ReadData`.

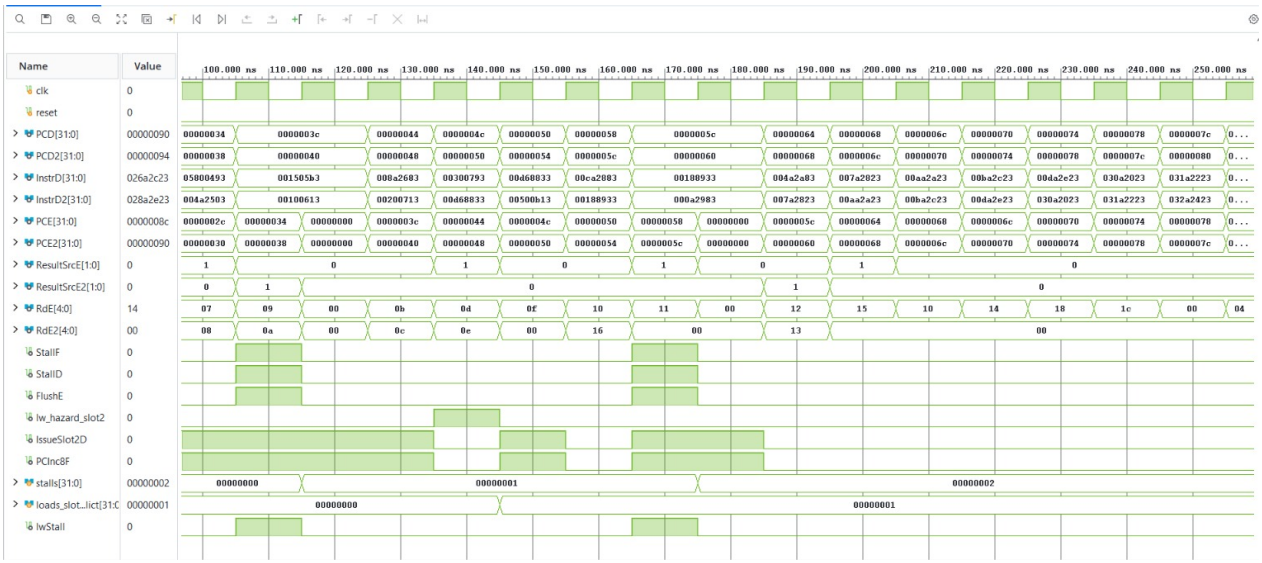


Figura 7: P7, 100–190 ns: Load-Use desde EX2, caso LU2 y `lw` con consumidor en el mismo par. Señales: `InstrD`, `InstrD2`, `ResultSrcE`, `ResultSrcE2`, `RdE`, `RdE2`, `lwStall`, `StallF`, `StallD`, `FlushE`, `lw_hazard_slot2`, `IssueSlot2D`.

5.7. P8: branches en ambas posiciones

`prog8_dual_branch`

c	t(ns)	Vía 1 en ID	Vía 2 en ID	Decisión	Eventos	Forwarding (par en EX)
0	30	00 addi x20, x0, 64	04 addi x1, x0, 4	DUAL		
1	40	08 addi x2, x0, 4	0c addi x3, x0, 0	DUAL		
2	50	10 beq x1, x2, t1	14 addi x3, x3, 100	DUAL		
3	60	18 addi x3, x3, 200	1c t1: addi x3, x3, 1	Vía 1 (RAW)	PCSrc FlushD FlushM2	A1=WB2 B1=MEM1 A2=MEM2
4	70	00 (burbuja)	00 (burbuja)	-		
5	80	1c t1: addi x3, x3, 1	20 beq x1, x0, n1	DUAL		
6	90	24 n1: beq x1, x0, n2	28 n2: addi x3, x3, 2	DUAL		
7	100	2c addi x3, x3, 4	30 beq x1, x2, t2	DUAL		A2=MEM1
8	110	34 addi x3, x3, 400	38 addi x3, x3, 500	Vía 1 (RAW)	PCSrc FlushD	A1=MEM2
9	120	00 (burbuja)	00 (burbuja)	-		
10	130	3c t2: beq x1, x0, n3	40 n3: beq x1, x2, t3	Vía 1 (CTRL)		
11	140	40 n3: beq x1, x2, t3	44 addi x3, x3, 600	DUAL		
12	150	48 t3: jal x5, t4	4c addi x3, x3, 700	Vía 1 (CTRL)	PCSrc FlushD FlushM2	
13	160	00 (burbuja)	00 (burbuja)	-		
14	170	48 t3: jal x5, t4	4c addi x3, x3, 700	Vía 1 (CTRL)		
15	180	4c addi x3, x3, 700	50 addi x3, x3, 800	Vía 1 (RAW)	PCSrc FlushD FlushM2	
16	190	00 (burbuja)	00 (burbuja)	-		
17	200	54 t4: addi x3, x3, 8	58 jal x6, t5	DUAL		
18	210	5c addi x3, x3, 900	60 addi x3, x3, 1000	Vía 1 (RAW)	PCSrc FlushD	
19	220	00 (burbuja)	00 (burbuja)	-		
20	230	64 t5: addi x7, x0, 3	68 addi x8, x0, 0	DUAL		
21	240	6c loop: beq x7, x0, end	70 add x8, x8, x7	DUAL		
22	250	74 addi x7, x7, -1	78 jal x0, loop	DUAL		A1=MEM1 A2=MEM2 B2=MEM1
23	260	7c addi x8, x8, 1000	80 end: sw x3, 0(x20)	DUAL	PCSrc FlushD	A1=WB1
24	270	00 (burbuja)	00 (burbuja)	-		
25	280	6c loop: beq x7, x0, end	70 add x8, x8, x7	DUAL		

c	t(ns)	Vía 1 en ID	Vía 2 en ID	Decisión	Eventos	Forwarding (par en EX)
26	290	74 addi x7, x7, -1	78 jal x0, loop	DUAL		
27	300	7c addi x8, x8, 1000	80 end: sw x3, 0(x20)	DUAL	PCSrc FlushD	
28	310	00 (burbuja)	00 (burbuja)	-		
29	320	6c loop: beq x7, x0, end	70 add x8, x8, x7	DUAL		
30	330	74 addi x7, x7, -1	78 jal x0, loop	DUAL		
31	340	7c addi x8, x8, 1000	80 end: sw x3, 0(x20)	DUAL	PCSrc FlushD	
32	350	00 (burbuja)	00 (burbuja)	-		
33	360	6c loop: beq x7, x0, end	70 add x8, x8, x7	DUAL		
34	370	74 addi x7, x7, -1	78 jal x0, loop	DUAL	PCSrc FlushD FlushM2	
35	380	00 (burbuja)	00 (burbuja)	-		
36	390	80 end: sw x3, 0(x20)	84 sw x5, 4(x20)	Vía 1 (MEM)		
37	400	84 sw x5, 4(x20)	88 sw x6, 8(x20)	Vía 1 (MEM)		
38	410	88 sw x6, 8(x20)	8c sw x8, 12(x20)	Vía 1 (MEM)		
39	420	8c sw x8, 12(x20)	90 sw x7, 16(x20)	Vía 1 (MEM)		
40	430	90 sw x7, 16(x20)	94 done: beq x0, x0, done	DUAL		
41	440	98 (burbuja)	9c (burbuja)	-	PCSrc FlushD	
42	450	00 (burbuja)	00 (burbuja)	-		

Tabla 12: Traza de ejecución de P8

Cada instrucción que debería anularse suma entre 100 y 1000 a `x3`; si algún *flush* fallara, `M[64]` sería distinto de 15. Los casos principales son:

- **Ciclo 3 (60 ns), `beq` tomado en la vía 1.** `PCSrcE1 = 1`: se activan `FlushD`, `FlushE` y `FlushM2`, que anula el `addi x3, x3, 100` de la vía 2 del mismo par.
- **Ciclos 5 y 6.** `beq` no tomado en la vía 2 y en la vía 1: la otra instrucción del par se ejecuta normalmente.
- **Ciclo 8 (110 ns), `beq` tomado en la vía 2.** `CtrlE2 = 1` y `PCSrcE2 = 1`. El `addi x3, x3, 4` de la vía 1 del mismo par se conserva (`FlushM2 = 0`).
- **Ciclo 10.** Dos `beq` en el mismo par: conflicto CTRL, el segundo se reagrupa.
- **Ciclos 14–15, `jal` en la vía 1.** La vía 2 no se emite (CTRL) y `x5` recibe `0x4C`.
- **Ciclos 17–18, `jal` en la vía 2.** El `addi x3, x3, 8` de la vía 1 se conserva y `x6` recibe `0x5C`.
- **Ciclos 21–34.** Bucle de tres iteraciones con `beq` de salida en la vía 1 y `jal x0, loop` en la vía 2; `x8` acumula $3 + 2 + 1 = 6$.

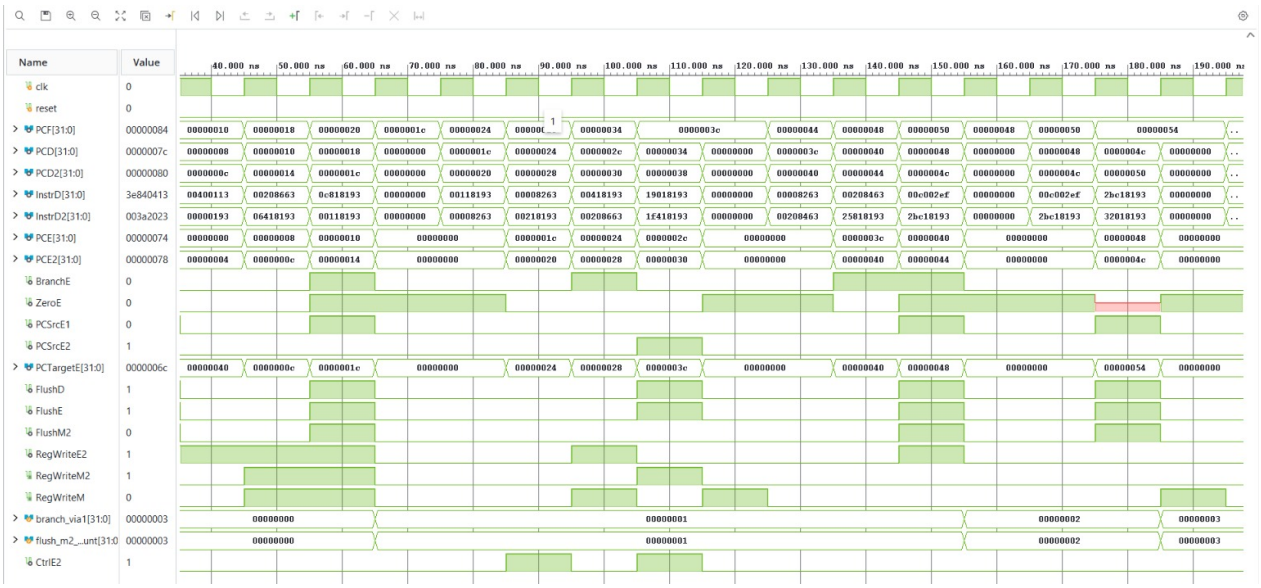


Figura 8: P8, 40–90 ns: `beq` tomado en la vía 1 y anulación de la vía 2 del par. Señales: `PCF`, `PCE`, `PCE2`, `BranchE`, `ZeroE`, `PCSrcE1`, `PCSrcE2`, `CtrlE2`, `PCTargetE`, `FlushD`, `FlushE`, `FlushM2`, `RegWriteM2`.

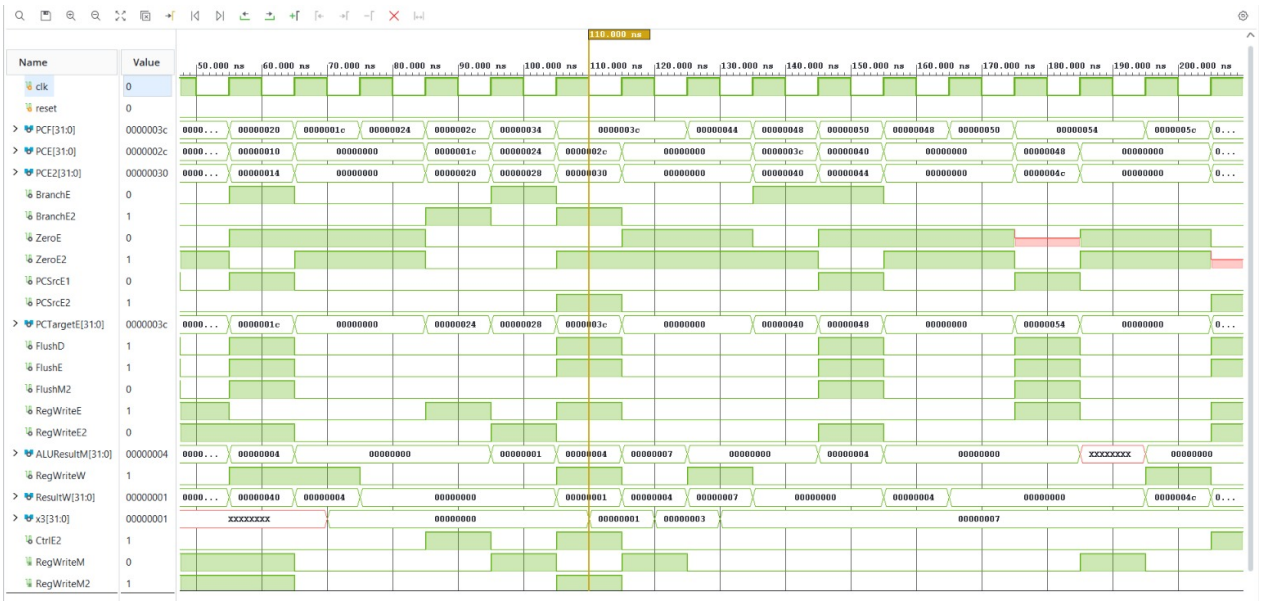


Figura 9: P8, 90–140 ns: `beq` tomado en la vía 2 con la vía 1 conservada. Mismas señales que la figura anterior, más `RegWriteM` y `ALUResultM`.

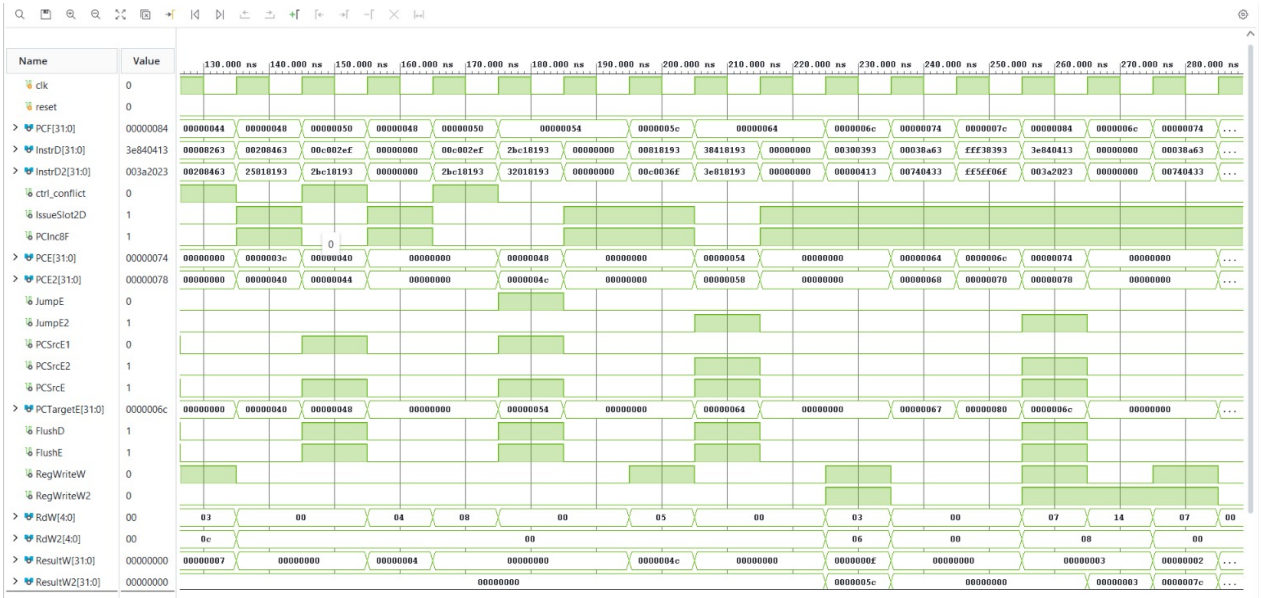


Figura 10: P8, 150–240 ns: `jal` en la vía 1 y en la vía 2. Señales: `InstrD`, `InstrD2`, `ctrl_conflict`, `IssueSlot2D`, `JumpE`, `JumpE2`, `PCSrcE`, `PCTargetE`, `ResultW`, `ResultW2`, `RdW`, `RdW2`.

5.8. P9: reagrupamiento y orden del programa

`prog9_dual_regroup`

c	t(ns)	Vía 1 en ID	Vía 2 en ID	Decisión	Eventos	Forwarding (par en EX)
0	30	00 addi x20, x0, 64	04 addi x1, x0, 1	DUAL		
1	40	08 addi x9, x0, 0	0c addi x2, x0, 0	DUAL		
2	50	10 add x1, x1, x1	14 addi x1, x1, 1	Vía 1 (RAW)		
3	60	14 addi x1, x1, 1	18 add x1, x1, x1	Vía 1 (RAW)		A1=WB2 B1=WB2
4	70	18 add x1, x1, x1	1c addi x1, x1, 1	Vía 1 (RAW)		A1=MEM1 B1=MEM1
5	80	1c addi x1, x1, 1	20 addi x9, x9, 1	DUAL		A1=MEM1 B1=MEM1
6	90	24 add x1, x1, x1	28 addi x1, x1, 1	Vía 1 (RAW)		A1=MEM1 B1=MEM1 B2=MEM1
7	100	28 addi x1, x1, 1	2c addi x2, x2, 5	DUAL		A1=MEM1 B1=MEM1
8	110	30 add x1, x1, x1	34 addi x9, x9, 1	DUAL		A1=MEM1 B1=MEM1
9	120	38 addi x9, x9, 1	3c addi x1, x1, 3	DUAL		A1=MEM1 B1=MEM1 B2=MEM1
10	130	40 addi x10, x0, 1	44 addi x10, x0, 2	DUAL		A1=MEM2 B1=MEM1 A2=MEM1
11	140	48 add x1, x1, x1	4c addi x9, x9, 1	DUAL		B1=MEM2
12	150	50 add x1, x1, x10	54 addi x2, x2, 7	DUAL		A1=WB2 B1=WB2 A2=WB1 B2=WB2
13	160	58 add x1, x1, x1	5c addi x9, x9, 1	DUAL		A1=MEM1 B1=WB2
14	170	60 addi x1, x1, -1	64 addi x9, x9, 1	DUAL		A1=MEM1 B1=MEM1 A2=WB2 B2=MEM1
15	180	68 sw x1, 0(x20)	6c sw x9, 4(x20)	Vía 1 (MEM)		A1=MEM1 A2=MEM2 B2=MEM1
16	190	6c sw x9, 4(x20)	70 sw x2, 8(x20)	Vía 1 (MEM)		B1=MEM1
17	200	70 sw x2, 8(x20)	74 sw x10, 12(x20)	Vía 1 (MEM)		B1=WB2
18	210	74 sw x10, 12(x20)	78 done: beq x0, x0, done	DUAL		
19	220	7c (burbuja)	80 (burbuja)	-	PCSrc FlushD	
20	230	00 (burbuja)	00 (burbuja)	-		

Tabla 13: Traza de ejecución de P9

La cadena sobre `x1` alterna “duplicar” y “sumar”, por lo que el resultado final ($x1 = 135$) solo se obtiene si cada instrucción se ejecuta exactamente una vez y en orden; una pérdida, una repetición o un intercambio darían otro valor. El contador `x9` debe valer exactamente 6.

Entre los ciclos 2 y 4 hay tres emisiones parciales seguidas (RAW): en cada una, la instrucción de la vía 2 aparece en el ciclo siguiente en la vía 1 y `PCD` avanza de 4 en 4 (`0x10`, `0x14`, `0x18`, `0x1C`), de modo que los pares pasan a comenzar en direcciones que no son múltiplos de 8. En el ciclo 5 el par vuelve a ser dual y en el ciclo 6 hay una nueva emisión parcial. Cada instrucción de la cadena recibe `x1` de MEM1 o WB2 según quién la produjo. En el ciclo 10 el par `addi x10, x0, 1 | addi x10, x0, 2` escribe el mismo registro en el mismo ciclo de WB (160 ns) y queda `x10 = 2`, que es el valor que usa `add x1, x1, x10`.

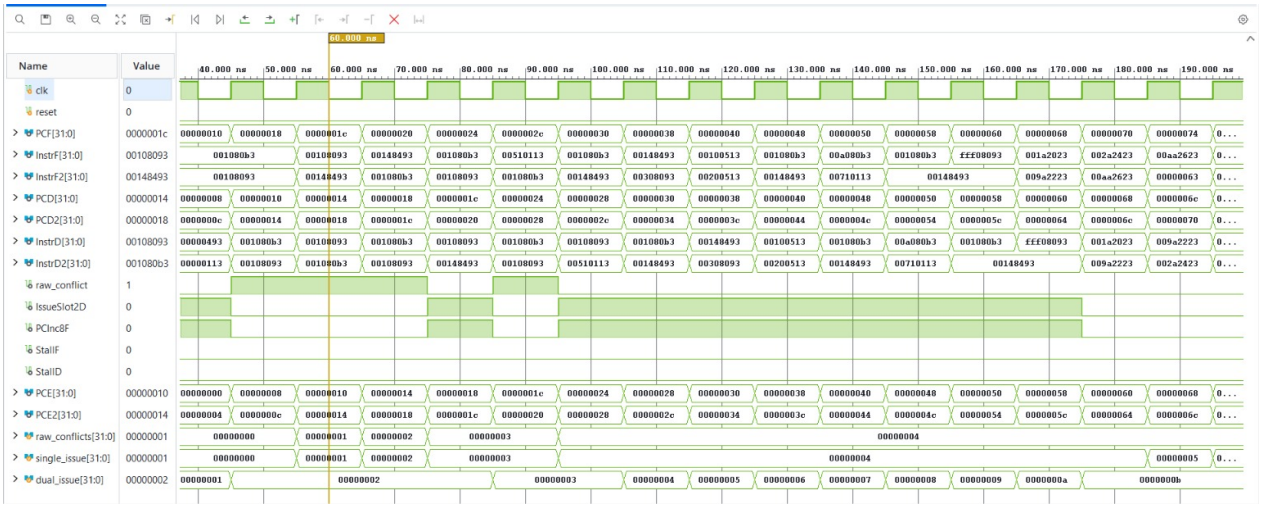


Figura 11: P9, 40–110 ns: emisiones parciales consecutivas. Señales: `PCF`, `InstrF`, `InstrF2`, `InstrD`, `InstrD2`, `PCD`, `PCD2`, `raw_hazard`, `IssueSlot2D`, `PCInc8F`.

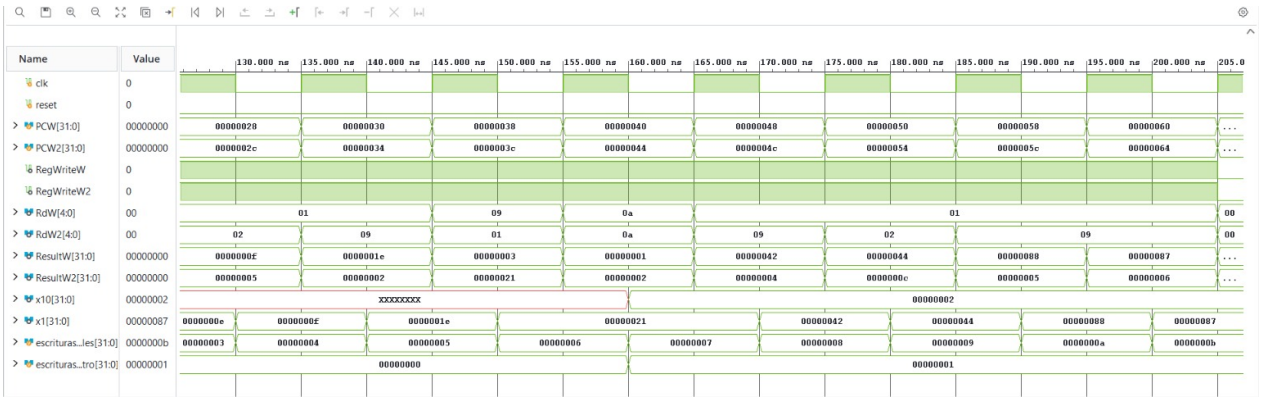


Figura 12: P9, 130–170 ns: dos escrituras al mismo registro. Señales: `RegWriteW`, `RegWriteW2`, `RdW`, `RdW2`, `ResultW`, `ResultW2`, `rf[10]`.

6. Comparación preliminar de ciclos e IPC

Se compararon el Single-Issue de E1 y el Dual-Issue de E2 sobre los nueve programas. Las instrucciones retiradas (N) excluyen el bucle `done`; los ciclos se cuentan, como en E1, desde el primer Fetch hasta que la última instrucción útil completa WB. Se definen IPC $= N/\text{ciclos}$, CPI $= \text{ciclos}/N$ y speedup $= \text{ciclos}_{E1}/\text{ciclos}_{E2}$.

		Single-Issue (E1)			Dual-Issue (E2)			
Programa	N	**Ciclos**	**IPC**	**CPI**	**Ciclos**	**IPC**	**CPI**	**Speedup**
P1 ISA	36	40	0.90	1.11	28	1.29	0.78	1.43×
P2 Forwarding	30	36	0.83	1.20	26	1.15	0.87	1.38×
P3 Stalling	28	39	0.72	1.39	31	0.90	1.11	1.26×
P4 Flushing	28	44	0.64	1.57	32	0.88	1.14	1.38×
P5 Dual ALU	26	30	0.87	1.15	21	1.24	0.81	1.43×
P6 Dual RAW	24	28	0.86	1.17	20	1.20	0.83	1.40×
P7 Dual MEM	38	44	0.86	1.16	32	1.19	0.84	1.38×
P8 Dual Branch	36	58	0.62	1.61	44	0.82	1.22	1.32×
P9 Regroup	30	34	0.88	1.13	22	1.36	0.73	1.55×
Total	276	353	0.78	1.28	256	1.08	0.93	1.38×

Tabla 14: Ciclos, IPC, CPI y speedup del Dual-Issue respecto al Single-Issue.

			Motivo de emisión parcial					
Programa	**Pares duales**	**Parciales**	**RAW**	**MEM**	**CTRL**	**LU2**	**Stalls E1 / E2**	**Saltos tomados**
P1 ISA	12	13	0	13	0	0	0 / 0	0
P2 Forwarding	11	10	3	7	0	0	0 / 0	1
P3 Stalling	10	10	2	8	0	0	5 / 6	1
P4 Flushing	14	3	0	3	0	0	0 / 0	6
P5 Dual ALU	9	9	0	9	0	0	0 / 0	0
P6 Dual RAW	8	9	1	8	0	0	0 / 0	0
P7 Dual MEM	12	15	1	13	0	1	2 / 2	0
P8 Dual Branch	17	6	0	4	2	0	0 / 0	9
P9 Regroup	12	7	4	3	0	0	0 / 0	0

Tabla 15: Desglose de la emisión en E2. Solo se cuentan los pares efectivamente emitidos (sin stall ni flush en ID); los saltos tomados excluyen el bucle final.

6.1. Análisis

El Dual-Issue reduce los ciclos en todos los programas, con speedups entre 1.26× y 1.55× (1.38× en el total) y un IPC medio de 1.08 frente a 0.78. El límite teórico, con dos instrucciones por ciclo y sin riesgos, sería un IPC cercano a 2; las causas de la diferencia se ven en la Tabla 15:

- **Puerto único de memoria.** Es la causa más frecuente de emisión parcial (68 de las 82 emisiones parciales). Los programas guardan sus resultados con secuencias de `sw` consecutivos que se emiten de a uno, y en ese tramo el procesador se comporta como un Single-Issue.
- **Saltos.** La penalización de 2 ciclos por salto tomado es la misma que en E1, pero en E2 cada ciclo perdido equivale a dos ranuras de emisión. Por eso los programas con más saltos tomados (P8 con 9, P4 con 6) tienen el menor IPC (0.82 y 0.88) y P8 el menor speedup entre los programas de E2 (1.32×).
- **Dependencias RAW.** Las cadenas de dependencias impiden emitir en pareja; aun así, P9, diseñado con una cadena larga, obtiene el mayor speedup (1.55×) porque las instrucciones independientes intercaladas se emiten junto a la cadena.
- **Load-Use.** El caso LU2 evita un stall en P7. En P3, en cambio, el Dual-Issue tiene un stall

más que el Single-Issue (6 frente a 5): el `nop` que separaba un `lw` de su consumidor se emite en el mismo ciclo que el `lw`, de modo que el consumidor queda a un solo ciclo de distancia y el stall que antes se evitaba vuelve a aparecer. Las separaciones con `nop` pensadas para un pipeline escalar dejan de ser efectivas en un Dual-Issue.

7. Conclusiones

- El procesador Dual-Issue In-Order está completo y es funcional: los nueve programas de prueba producen resultados satisfactorios y muestran la correcta funcionalidad del procesador.
- La Issue Unit decide la emisión con cuatro condiciones (RAW, MEM, CTRL y LU2) y conserva la instrucción no emitida desplazándola a la vía 1, con el invariante $\texttt{PCF} = \texttt{PCD2} + 4$. Esto preserva el orden del programa sin pérdidas ni duplicados, como verifica P9.
- El *forwarding* de 3 bits con prioridad MEM2 > MEM1 > WB2 > WB1 resuelve todas las dependencias entre pares, incluido el WAW dentro de un par, y el Register File con prioridad de la vía 2 mantiene la semántica secuencial.
- Los recursos compartidos (memoria de datos y unidad de saltos) se asignan con multiplexores simples, porque la Issue Unit garantiza que nunca los piden dos instrucciones a la vez. El *flush* selectivo anula la vía 2 solo cuando el salto está en la vía 1.
- El speedup preliminar es de 1.26× a 1.55× (IPC de 0.78 a 1.08 en conjunto). El puerto único de memoria y la penalización de los saltos son los principales factores que limitan el IPC.